

Алгоритмизация и программирование

Лекция 2 (Go)

Базовые сведения

Go создан в Google в 2007 году тремя инженерами:

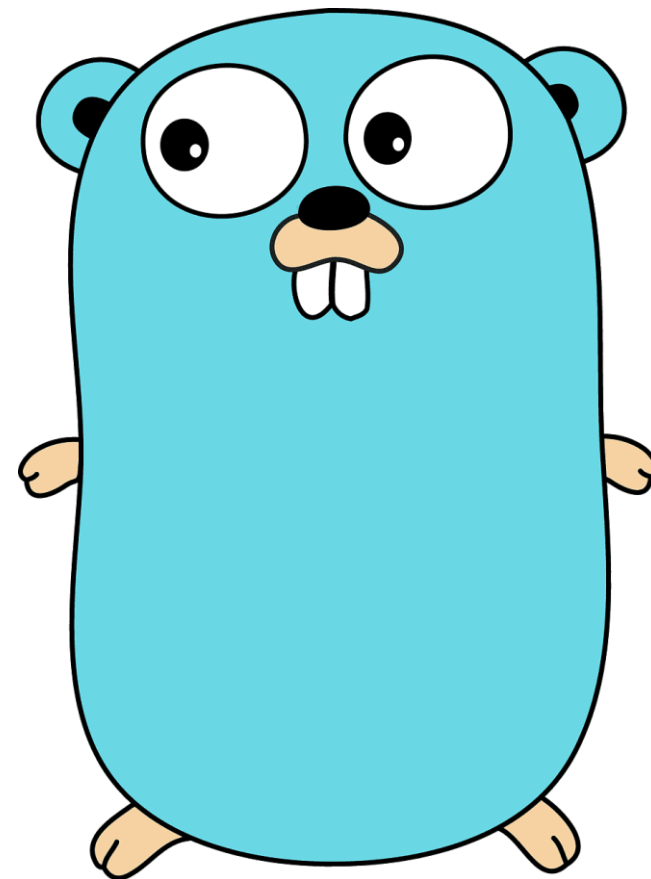
- Роб Пайк (Rob Pike) – соавтор ОС Plan 9, языка Limbo, один из создателей UTF-8.
- Кен Томпсон (Ken Thompson) – создатель языка B, соавтор ОС UNIX, лауреат премии Тьюринга (1983).
- Роберт Гриземер (Robert Griesemer) – работал над V8 (JavaScript-движок Chrome) и HotSpot (JVM).

21 сентября 2007 года авторы начали обсуждать новый язык на доске; 25 сентября Роб Пайк предложил имя «Go».

Публичный релиз с открытым исходным кодом – 10 ноября 2009 года.

Стабильная версия Go 1.0 – 28 марта 2012 года.

Талисман языка – суслик Gopher.



Характеристика языка Go

Мультипарадигменный язык: процедурное, объектного-ориентированное (но без классов), некоторые аспекты функционально программирования.

Типизация: статическая; сильная (строгая) + интерфейсы.

Язык компилируемый - компилируется в машинный код.

Управление памятью через сборку мусора (garbage collection), встроена в рантайм.

Минималистичный синтаксис: всего 25 ключевых слов;

Встроенная конкурентность: горутины (goroutines) и каналы (channels) как языковые примитивы, а не библиотечная надстройка.

Нет классов и наследования; вместо этого — композиция, структуры и интерфейсы.

Нет исключений: ошибки обрабатываются явно через возврат значения error.

Кроссплатформенность: Linux, macOS, Windows, embedded, WebAssembly.

Активно развивается командой Go Team (Google) и сообществом; выходит 2 релиза в год.

История и релизы

Дата	Версия	Ключевые события
21.09.2007	—	Начало обсуждения нового языка в Google
25.09.2007	—	Предложено имя «Go»
10.11.2009	—	Публичный релиз с открытым исходным кодом
28.03.2012	Go 1.0	Первый стабильный релиз, обещание совместимости
2015	Go 1.5	Компилятор и рантайм переписаны на Go
2018	Go 1.11	Модули (go modules) — новый механизм зависимостей
2022	Go 1.18	Дженерики (обобщённое программирование)
2024	Go 1.22	Улучшения в рантайме и стандартной библиотеке
2026	Go 1.27	Актуальный релиз на момент лекции

Спецификация и документация Go

Go не стандартизирован ISO/IEC или IEEE. Стандарт де-факто задаётся официальной спецификацией языка (The Go Programming Language Specification).

Спецификация доступна онлайн и обновляется с каждой версией языка: <https://go.dev/ref/spec>

Исходный код компилятора, рантайма и стандартной библиотеки: <https://github.com/golang/go>

Ключевой принцип эволюции — обратная совместимость (Go Compatibility Promise): программы, написанные для Go 1, продолжают собираться в новых версиях.

Дополнительные документы:

- Go Memory Model — правила видимости данных между горутинами.
- Effective Go — идиоматическое написание кода на Go.
- Go FAQ — ответы на частые вопросы о дизайне языка.

Первая модель программы на Go

Только один файл



Весь код программы
полностью в одном файле

Структура программы

Программа состоит строго из 3х секций:

```
package main
```

Имя пакета всегда `main`
Эта секция обязательна.

```
import "fmt"
```

В этой секции только подключаем стандартные библиотеки, если нужно.

```
func main() {  
    // весь код пишется здесь  
}
```

Весь код пишем только в этой секции.
Эта секция обязательна.

Минимальная программа

```
package main
```

```
func main() {  
}
```

Подключение нескольких библиотек

```
package main

import (
    "fmt"
    "math"
)

func main() {
    fmt.Println(math.Sqrt(25))
}
```

Используются только подключённые пакеты: иначе – ошибка компиляции.
Специальный импорт `_ "pkg"` — только для побочного эффекта (init).

Комментарии

Комментарии можно писать в любом месте файла. Они не влияют на результат работы программы.

```
// Однострочный комментарий: от // до конца строки
```

```
/*
```

```
    Многострочный комментарий.
```

```
    Не вкладываются друг в друга.
```

```
*/
```

```
func main() {
```

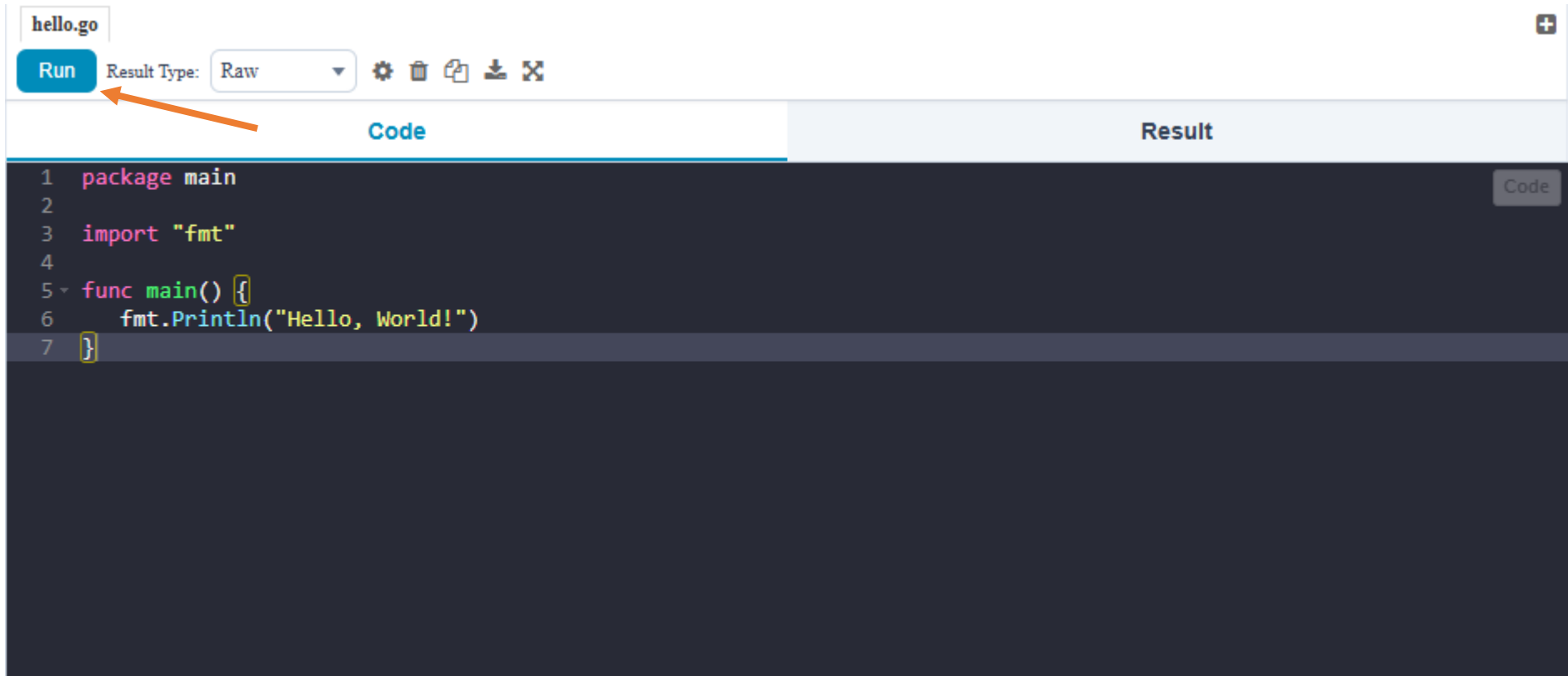
```
    fmt.Println("hi") // после кода
```

```
}
```

Применяются для документирования кода или чтобы временно отключить его часть без удаления.

Программа просто запускается

Мы не управляем процессом сборки программы и не думаем о нём. По нажатию кнопки происходит магия и программа просто запускается.



The screenshot shows the Go Playground interface. At the top, there is a tab labeled 'hello.go'. Below it, a blue 'Run' button is highlighted with an orange arrow. To the right of the 'Run' button is a 'Result Type' dropdown menu set to 'Raw', followed by several icons for settings, deletion, copying, downloading, and sharing. The main area is divided into two tabs: 'Code' (active) and 'Result'. The 'Code' tab contains the following Go code:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, World!")
7 }
```

A 'Code' button is visible in the top right corner of the code editor area.

Структура кода в main

```
func main() {  
    stmt1  
    stmt2  
    {  
        stmt3  
        stmt4  
    }  
    stmt5  
}
```

Базовая единица выполнения – стейтмент (statement).

Кроме стейтментов есть объявления (declarations): var, const, type, func.

Стейтмент (statement)

- В русскоязычной литературе встречаются переводы: «оператор», «инструкция», «команда».
- В Go, как и в C++, корректнее говорить «стейтмент».
- Программа на Go – последовательность стейтментов.
- Стейтмент – базовая единица выполнения программы.
- Стейтмент не возвращает значение (в отличие от выражения).

Виды стейтментов

Виды стейтментов в Go

- Declaration (объявление): var, const, type, func, import
- Labeled statement (метки)
- Simple statement:
 - Empty statement (пустой стейтмент)
 - Expression statement (стейтмент-выражение)
 - Send statement (отправка в канал): ch <- x
 - IncDec statement (инкремент/декремент). Только постфиксный вариант: i++, i--
 - Assignment statement (присваивание): =, +=, ...
 - Short Variable Declaration statement (только внутри функций): x := 1
- Go statement (запуск горутины): go f()
- Return statement
- Break / Continue / Goto / Fallthrough
- Compound statement (блок)
- If / Switch / Select / For
- Defer statement (отложенный вызов): defer f()

Empty Statement (Пустой стейтмент)

```
func main() {  
    ;  
}
```

Expression Statement (Стейтмент-выражение)

Ровно два вида **выражений** могут быть использованы в контексте где требуется выражение и как отдельные стейтменты (т.е. как самостоятельные единицы исполнения):

- Вызов функции (своих и некоторых встроенных) и вызов метода;
- Получение из канала (receive).

Всё. Больше ничего.

Все остальные **выражения** могут быть использованы только в составе других стетментов, но не как самостоятельные стейтменты.

Expression Statement – вызов функции/метода

Следующие **встроенные** функции и методы не разрешено использовать как стейтмент-выражение, только в составе каких-нибудь других стейтментов:

- append cap complex imag len make max min new real
- unsafe.Add unsafe.Alignof unsafe.Offsetof unsafe.Sizeof unsafe.Slice unsafe.SliceData unsafe.String unsafe.StringData

```
func f() int { return 42 }
```

```
type T struct{}
```

```
func (T) M() int { return 1 }
```

```
func main() {
```

```
    f()           // OK – это Expression statement
```

```
    len("foo")    // Не OK – функция-исключение. Результат не используется, поэтому код бесполезный
```

```
    var t T
```

```
    t.M()         // OK – это Expression statement
```

```
}
```

Встроенные функции и методы не указанные в списке, а также созданные пользователем, могут быть использованы как стейтмент-выражение.

Expression Statement – получение из канала

```
func main() {  
    ch := make(chan int, 1) // Short var decl statement  
    ch <- 5 // Send statement  
    <-ch    // OK – Expression statement  
}
```

Выражения но НЕ Expression Statement

```
x + y      // ошибка  
z = x + y  // OK. Assignment statement
```

```
len(s)      // ошибка (встроенная функция-исключение)  
if len(s) > 10 {} // OK. If statement
```

```
append(s, 1) // ошибка (встроенная функция-исключение)  
int(x)       // ошибка (преобразование типа)  
m[k]         // ошибка (индексация)  
p.x          // ошибка (селектор без вызова)
```

```
close(ch)    // OK (встроенная функция, но не исключение)  
delete(m, k) // OK (встроенная функция, но не исключение)  
panic("boom") // OK (встроенная функция, но не исключение)  
print("hi")   // OK (встроенная функция, но не исключение)  
println("hi") // OK (встроенная функция, но не исключение)  
copy(dst, src) // OK (встроенная функция, но не исключение)
```

Автоматическая вставка точек с запятой

В официальной грамматике Go точки с запятой являются терминаторами стейтментов (statement terminators). Но чтобы код был чистым, спецификация разрешает их опускать с помощью двух правил:

- Вставка на новой строке: когда лексер разбивает входной поток на токены, он автоматически вставляет ; в конце непустой строки, если её последний токен — один из следующих:
 - идентификатор (x, myVar);
 - литерал (целое, число с плавающей точкой, мнимое число, руна или строка);
 - одно из ключевых слов: break, continue, fallthrough, return;
 - один из операторов и разделителей: ++, --,),], }.
- Пропуск перед закрывающей скобкой: чтобы сложные стейтменты можно было разместить в одной строке, точку с запятой можно опустить непосредственно перед закрывающей) или }

Автоматическая вставка точек с запятой

```
func main() { // OK
    // ...
}

func main() // НЕ ОК. После ) будет ;
{
    // ...
}

func (T) M() int { return 1 } // ОК. Перед } после return 1 вставится ;

longStr := "Hello World! " // НЕ ОК. После литерала будет ;
    + "How are you?"

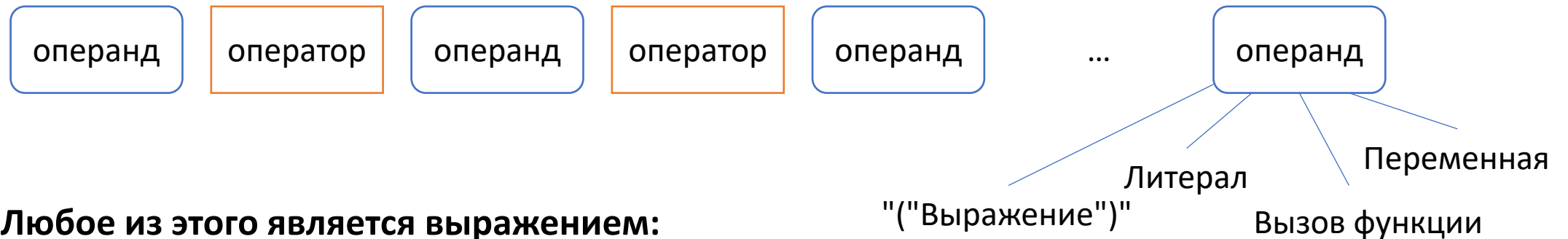
longStr := "Hello World! " + // ОК
    "How are you?"

fmt.Printf("[%s] %d %d %v" // НЕ ОК. После литерала будет ;
    , str, num1, num2, nameMap)

fmt.Printf("[%s] %d %d %v", // ОК
    str, num1, num2, nameMap)
```

Выражение (expression)

Выражение (expression) - определяет вычисление значения путем применения операторов и функций к операндам.



Любое из этого является выражением:

- Отдельная переменная: x (возвращает своё значение);
- Литерал: 42 (возвращает 42);
- Вызов функции: $\text{sqrt}(25)$ (возвращает 5.0);
- Комбинация всего вышеперечисленного с операторами: $(a + b) * 2 - \text{func}()$.

Для бинарных операторов типы операндов должны быть **идентичными**, за исключением случаев, когда операция включает сдвиги или нетипизированные константы.

Литералы

```
Literal    = BasicLit | CompositeLit | FunctionLit .  
BasicLit   = int_lit | float_lit | imaginary_lit | rune_lit | string_lit .
```

BasicLit:

```
42          // целочисленный литерал (int)  
3.14159     // литерал с плавающей точкой (float64)  
1e6         // экспоненциальная форма  
2.71828i    // комплексный литерал (imaginary literal)  
1.e+0i      // комплексный литерал (imaginary literal)  
6.67428e-11i // комплексный литерал (imaginary literal)  
'A'        // руна (rune = int32)  
"Hello"     // строковый литерал (string)  
true        // булев литерал  
nil         // нулевое значение для указателей/срезов/мап/каналов  
0b1010      // двоичный  
0o777       // восьмеричный  
0xDEADBEEF  // шестнадцатеричный
```

Операторы

Операторы объединяют операнды в выражения.

```
binary_op  = "||" | "&&" | rel_op | add_op | mul_op .
rel_op     = "==" | "!=" | "<" | "<=" | ">" | ">=" .
add_op     = "+" | "-" | "|" | "^" .
mul_op     = "*" | "/" | "%" | "<<" | ">>" | "&" | "&^" .

unary_op   = "+" | "-" | "!" | "^" | "*" | "&" | "<-" .
```

Тернарного оператора в Go нет.

Приоритет и ассоциативность операторов

Унарные операторы имеют наивысший приоритет. Поскольку ++ и --, не операторы, а стейтменты применяемые к выражению, они не входят в иерархию операторов. Следовательно, выражение *p++ эквивалентно (*p)++. Присваивание, тоже не является оператором, а стейтментом.

Существует пять уровней приоритета бинарных операторов:

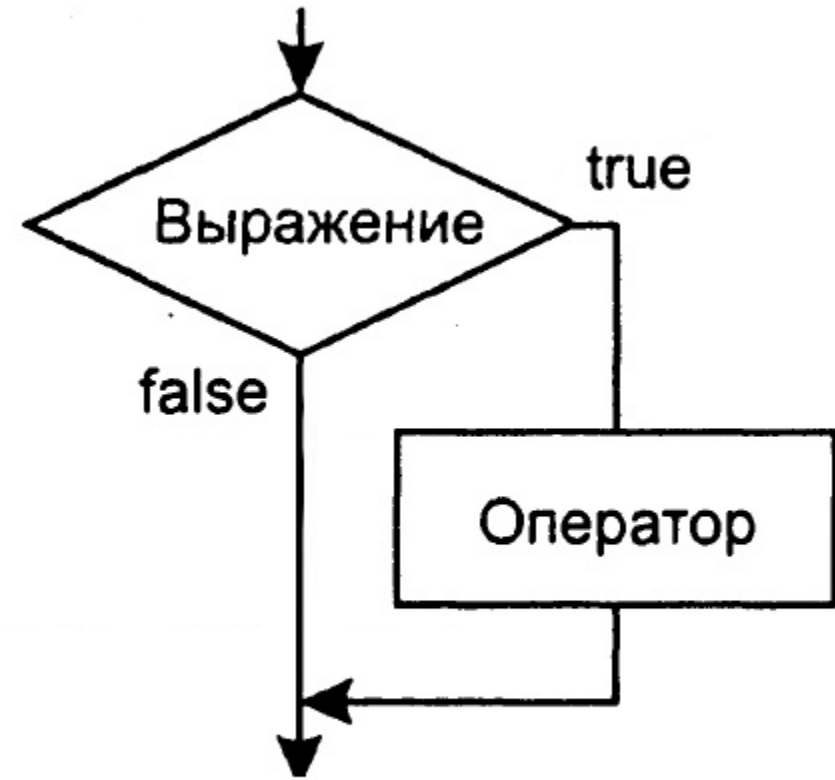
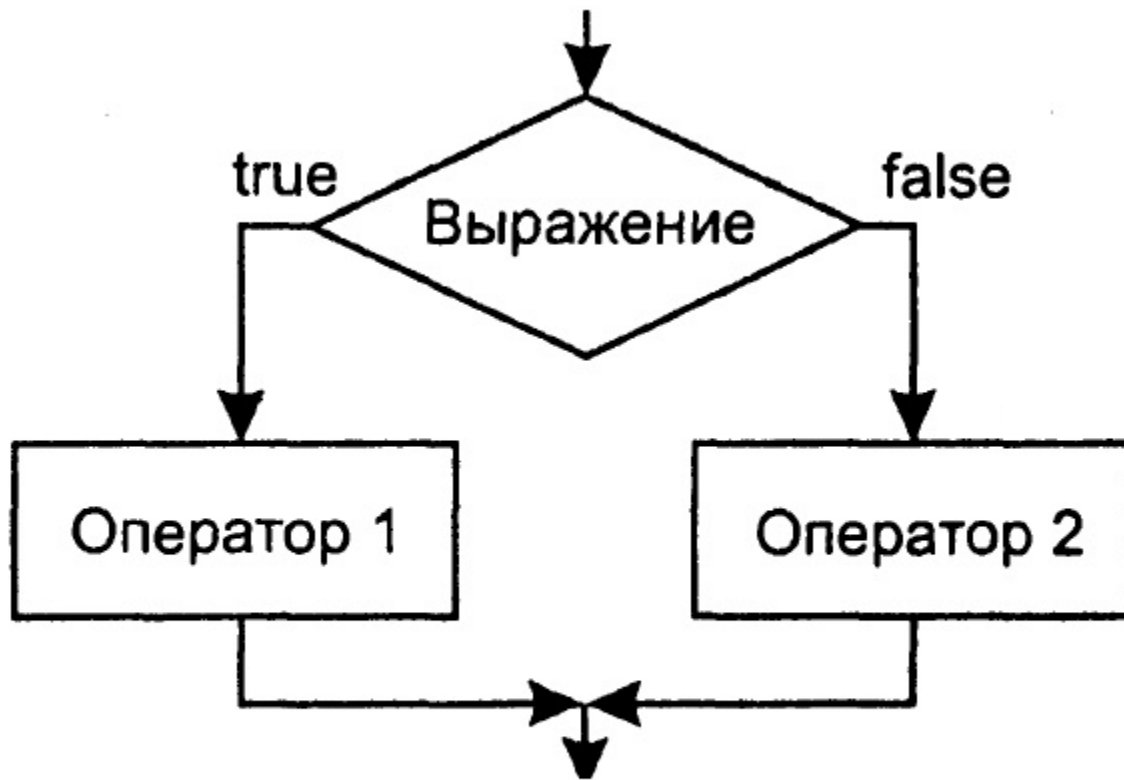
Приоритет	Оператор
5	* / % << >> & &^
4	+ - ^
3	== != < <= > >=
2	&&
1	

Ассоциативность: бинарные операторы с одинаковым приоритетом выполняются слева направо

Expression

<code>+x</code>	<code>// x</code>
<code>42 + a - b</code>	<code>// (42 + a) - b</code>
<code>23 + 3*x[i]</code>	<code>// 23 + (3 * x[i])</code>
<code>x <= f()</code>	<code>// x <= f()</code>
<code>^a >> b</code>	<code>// (^a) >> b</code>
<code>f() g()</code>	<code>// f() g()</code>
<code>x == y+1 && <-chanInt > 0</code>	<code>// (x == (y+1)) && ((<-chanInt) > 0)</code>

Условный оператор



Структурная схема условного оператора

Тип bool

```
var x bool = true    // Истина  
var y bool = false   // Ложь
```

* Маленькими буквами

Преобразование типа bool

В Go нет возможности преобразовать bool в другой тип или другой тип в bool стандартным способом. Автоматического преобразования тоже нет.

Преобразование типа bool руками

```
package main

import "fmt"

func main() {
    var a int = 10
    var b bool = a != 0
    fmt.Println(b) // true

    var c int
    if b {
        c = 1
    } else {
        c = 0
    }
    fmt.Println(c) // 1
}
```

Откуда берётся true и false

Операторы сравнения

Оператор	Проверяет на	Пример использования	Результат
==	Равенство	3 == 7	false
!=	Неравенство	3 != 7	true
<	Меньше	3 < 7	true
>	Больше	3 > 7	false
<=	Меньше или равно	7 <= 7	true
>=	Больше или равно	7 >= 7	true

Логические операторы

Название	Как выглядит	Как использовать
И	&&	a && b
ИЛИ		a b
НЕ	!	!a

a	b	a && b	a b	! a
false	false	false	false	true
true	false	false	true	false
false	true	false	true	true
true	true	true	true	false

Логические операторы применяются только к операндам типа `bool`.

Операторы И и ИЛИ вычисляются по сокращённым правилам, т.к. если результат можно получить вычислив первый аргумент, второй не вычисляется:

```
func plus() bool{  
    fmt.Println("+")  
    return true  
}
```

```
fmt.Println(true && plus()) // +true  
fmt.Println(false && plus()) // false
```

Логические операторы. Возможные ошибки

```
int a = 10;
```

```
-1 < a < 2    // ошибка
```

Последнее выражение вычисляется последовательно:

```
-1 < a < 2
```

```
(-1 < a) < 2
```

```
true < 2    // ошибка. Число не типа bool
```

Если хотите получить результат по математическим правилам пишите:

```
(-1 < a) && (a < 2)    // false
```

if

Ключевое слово **если**

Выражение

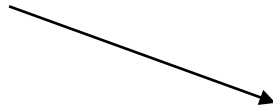
Тело (Стэйтмент Блок)

if

a > b {

fmt.Println("Hello")

}



if выражение

Ожидается, что выражение после ключевого слова `if` типа `bool`.

If инициализация; проверка

SimpleStmt
↓

```
if x := f(); x < y {  
    return x  
} else if x > z {  
    return z  
} else {  
    return y  
}
```

if-else

Выражение



Ключевое слово **если** → `if a > b {`
 `fmt.Print("Hello")`
Ключевое слово **иначе** → `}else {`
 `fmt.Print("Bye")`
 `}`



Тело **if**



Тело **else**

if

```
if /* условие */ {  
    /* true */  
}
```

If-else

```
if /* условие */ {  
    /* true */  
}else{  
    /* false */  
}
```

If-else if

```
if /* условие 1 */ {  
    /* true */  
} else if /* условие 2 */ {  
    /* true */  
} else {  
    /* false */  
}
```

Вложенный if

```
if /* условие */ {  
    if /* условие */ {  
        /* true */  
    }else{  
        /* false */  
    }  
}else{  
    if /* условие */ {  
        /* true */  
    }else{  
        /* false */  
    }  
}
```

Тернарный оператор

Тернарного оператора или аналогов в Go нет.

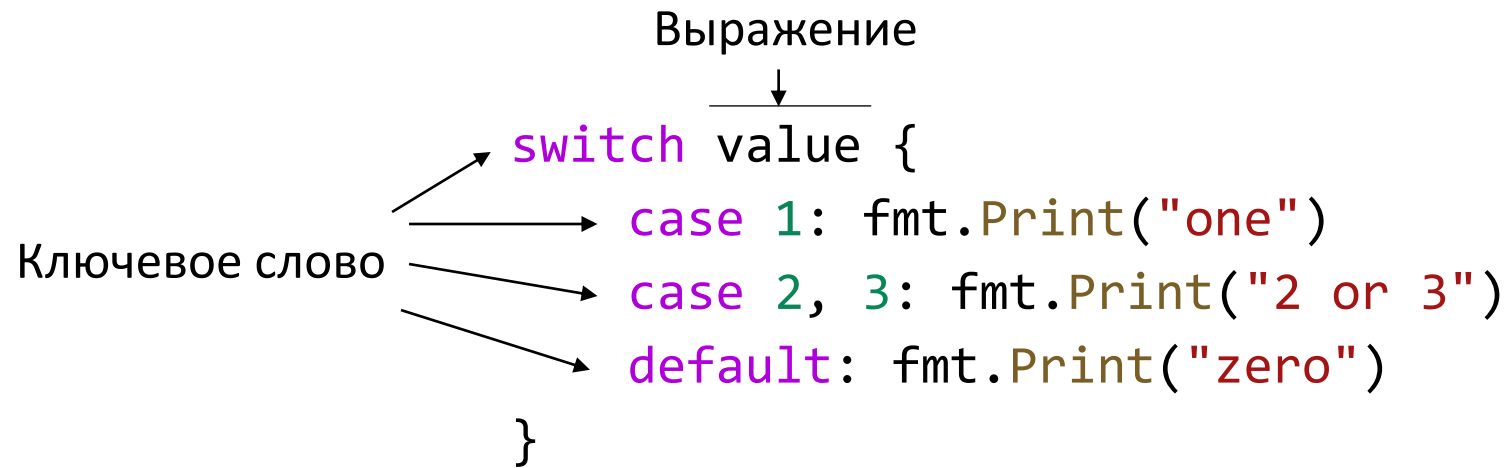
switch ExprSwitchStmt | TypeSwitchStmt

Существует две формы switch по выражению и по типу:

- В switch по выражению варианты содержат выражения, которые сравниваются со значением выражения-переключателя.
- В switch по типу варианты содержат типы, которые сравниваются с типом специально аннотированного выражения-переключателя.

Выражение-переключатель вычисляется в операторе switch ровно один раз.

switch ExprSwitchStmt (по выражению)



switch выражение

Выражение может быть любого типа, но обязательно такого же как и в case - ах.

Если выражение отсутствует, то это тоже самое, что и **true**.

switch - fallthrough

```
nextstop := "B"
fmt.Println("Stops ahead of us:")

switch nextstop {

case "A":
    fmt.Println("A")
    fallthrough

case "B":
    fmt.Println("B")
    fallthrough

case "C":
    fmt.Println("C")
    fallthrough

case "D":
    fmt.Println("D")
    fallthrough

case "E":
    fmt.Println("E")
}
```

switch инициализация; выражение

SimpleStmt



```
switch res := a > b; res {  
    case true: fmt.Print(":")  
    default: fmt.Print(":(")  
}
```

switch TypeSwitchStmt (по типу)

```
switch i := x.(type) { // или просто x.(type)
    case nil:
        printString("x is nil") // type of i is type of x (interface{})
    case int:
        printInt(i) // type of i is int
    case float64:
        printFloat64(i) // type of i is float64
    case func(int) float64:
        printFunction(i) // type of i is func(int) float64
    case bool, string:
        printString("type is bool or string") // type of i is type of x (interface{})
    default:
        printString("don't know the type") // type of i is type of x (interface{})
}
```

goto

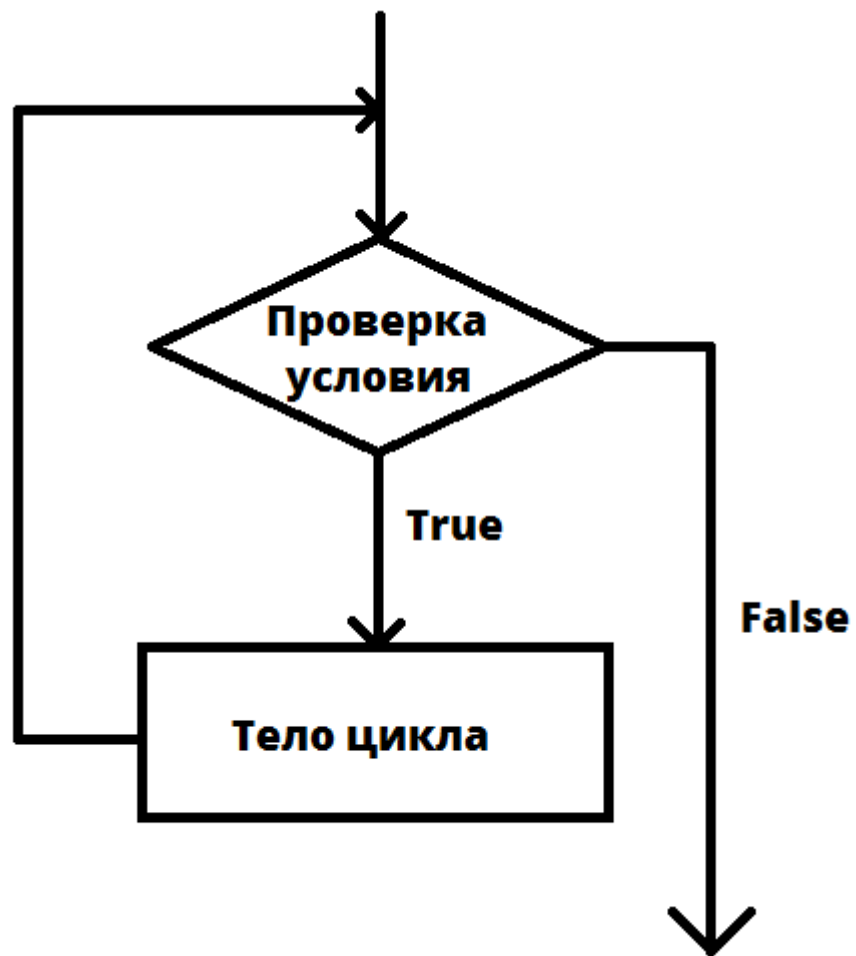
```
label:  
    /* код */  
goto label
```

* label – обычный идентификатор

Оператор цикла



Оператор цикла



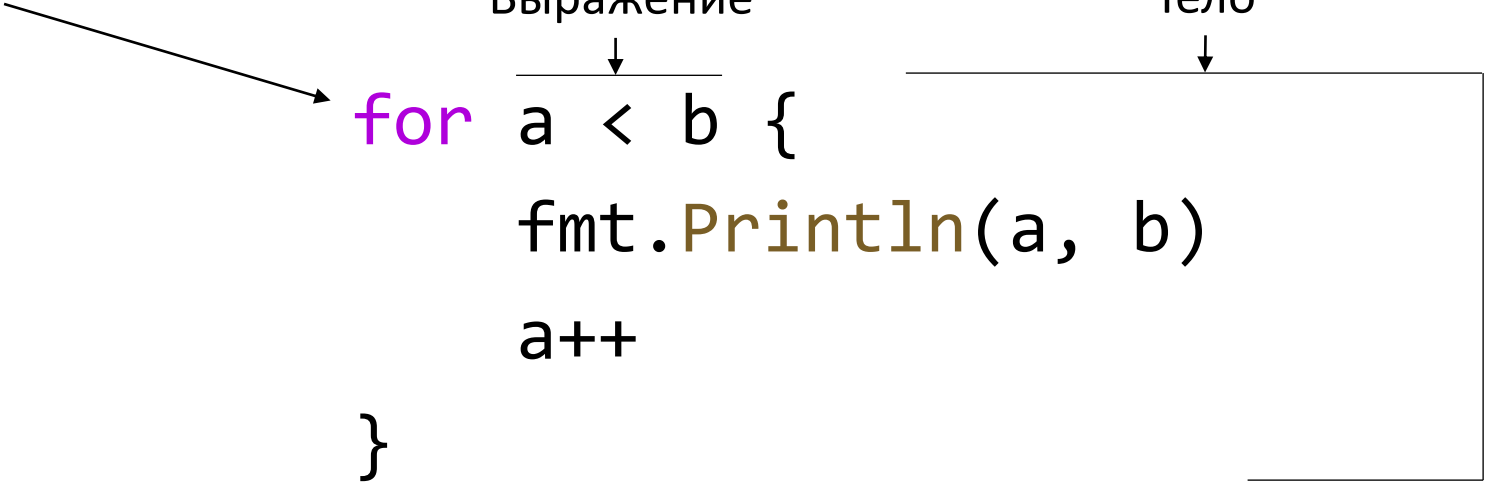
for

for

Ключевое слово

Выражение

Тело



```
for a < b {  
    fmt.Println(a, b)  
    a++  
}
```

The diagram illustrates the components of a `for` loop. The word `for` is highlighted in purple and is pointed to by an arrow from the label "Ключевое слово". The expression `a < b` is underlined and pointed to by an arrow from the label "Выражение". The body of the loop, consisting of `fmt.Println(a, b)` and `a++`, is enclosed in curly braces and pointed to by an arrow from the label "Тело".

for выражение

Ожидается, что выражение после ключевого слова `for` типа `bool`.

Бесконечный цикл

Ключевое слово

for

{

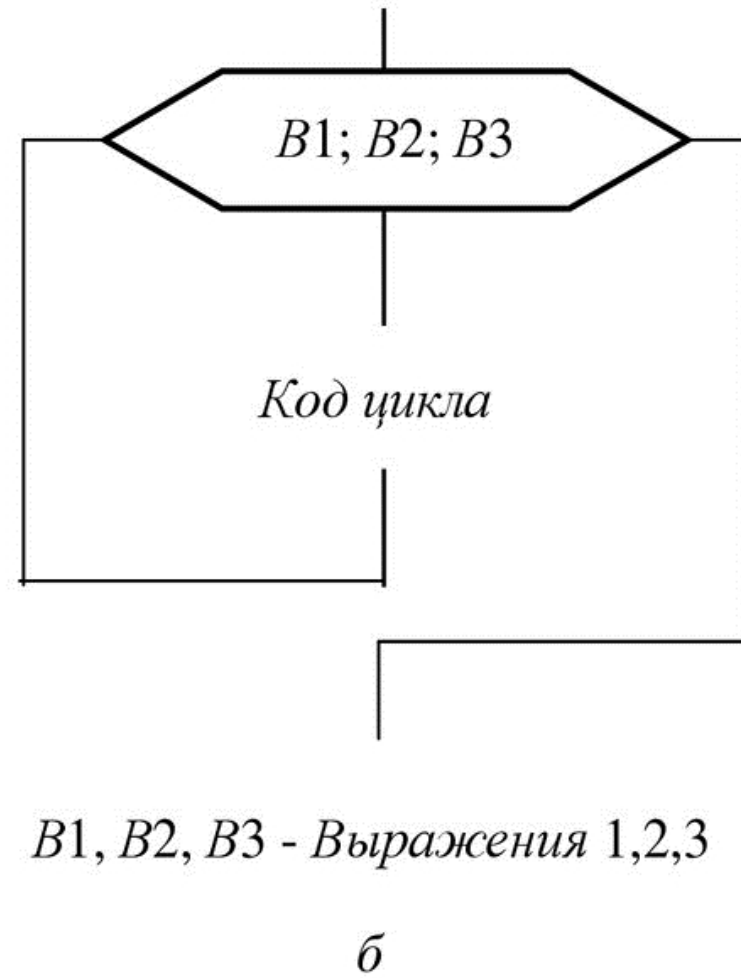
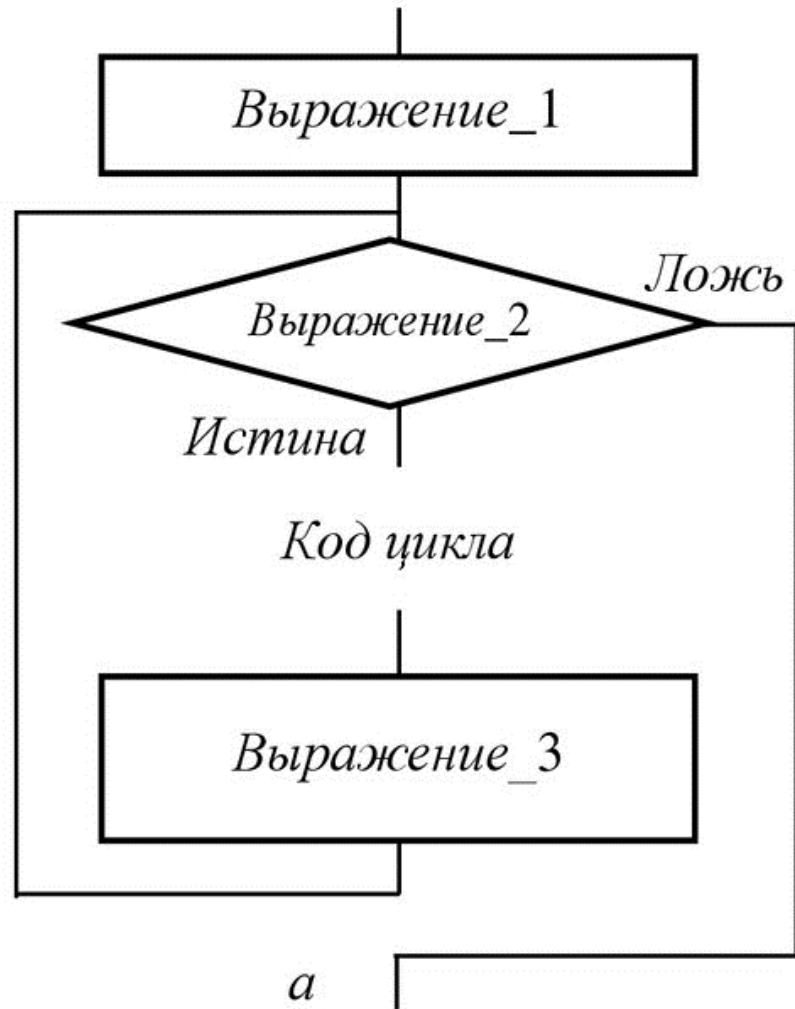
fmt.Println("+")

}

Тело



Оператор цикла for



for

Ключевое слово

SimpleStmt

Выражение

SimpleStmt

Тело

```
for count := 0; count <= 10; count++ {  
    fmt.Println("My counter is at", count)  
}
```

for SimpleStmt; Выражение; SimpleStmt

Каждое из выражение не обязательное (можно не писать), но точки с запятой писать нужно.

for RangeClause

```
for [ ExpressionList "=" | IdentifierList ":=" ] range Expression {  
    // Код  
}
```

for RangeClause

Range expression			1st value		2nd value		
array or slice	a	[n]E, *[n]E, or []E	index	i int	a[i]	E	
string	s	string type	index	i int		rune	
map	m	map[K]V	key	k K	m[k]	V	
channel	c	chan E, <-chan E	element	e E			
integer value	n	integer type, or untyped int	value	i			
function, 0 values	f	func(yield func() bool)					
function, 1 value	f	func(yield func(V) bool)	value	v V			yield cannot be variadic
function, 2 values	f	func(yield func(K, V) bool)	key	k K	v	V	yield cannot be variadic

for RangeClause пример - Массив (array)

```
var a [5]int
for i, v := range a {
    // тип i это int
    // тип v это int
    // v == a[i]
    // массив копируется в range-выражение (если только не используется указатель на массив), поэтому изменения a внутри цикла
    // не влияют на итерацию.
}
```

```
var a [5]int
for i := range a {
    // тип i это int
    // v отсутствует — цикл выдаёт только индексы 0..len(a)-1
    // сам массив при этом не индексируется
}
```

```
var p *[5]int = &a
for i, v := range p {
    // тип i это int
    // тип v это int
    // v == (*p)[i]
    // перебирается массив по указателю, копии массива нет
}
```

for RangeClause пример - Срез (slice)

```
var s []string = []string{"a", "b", "c"}
for i, v := range s {
    // тип i это int
    // тип v это string
    // v == s[i]
}
```

```
var s []string
for i := range s {
    // тип i это int
    // v отсутствует — только индексы
}
```

```
var s []int // s == nil
for range s {
    // тело не выполнится ни разу
}
```

for RangeClause пример - Строка (string)

```
s := "Привет, 世界!"
for i, r := range s {
    // тип i это int (байтовый индекс начала руны)
    // тип r это rune (int32) – кодовая точка Unicode
    // i – это индекс первого байта UTF-8-последовательности
    // r – значение соответствующей кодовой точки
    // при встрече некорректной UTF-8-последовательности r == 0xFFFFD (символ замены), а следующая итерация сдвигается на 1
    байт.
}

s := "\xff\xfe"
for i, r := range s {
    // r == 0xFFFFD (Unicode replacement character)
    // i = 0, потом i = 1 – сдвиг на один байт
}

s := "abc"
for i := range s {
    // тип i это int
    // только байтовые индексы начал рун
}
```

for RangeClause пример - Map

```
m := map[string]int{"a": 1, "b": 2}
for k, v := range m {
    // тип k это string
    // тип v это int
}
```

```
m := map[string]int{"a": 1}
for k := range m {
    // тип k это string
    // только ключи
    // Особенность: порядок обхода не определён и может меняться от итерации к итерации. nil-map даёт 0 итераций.
    // Особенность: если во время обхода удалить ещё не достигнутый элемент — он не будет выдан.
    // Особенность: если создать новый — он может быть выдан, а может быть пропущен
}
```

```
var m map[string]int // m == nil
for range m {
    // тело не выполнится ни разу
}
```

for RangeClause пример - Канал (channel)

```
ch := make(chan int, 3)
ch <- 1
ch <- 2
ch <- 3
close(ch)
```

```
for v := range ch {
    // тип v это int
    // v – последовательные значения, отправленные в канал
    // Только одна переменная итерации – значение из канала. Индекса нет.
}
```

```
var ch chan int // ch == nil
for v := range ch {
    // цикл заблокируется навсегда, итераций не будет
    _ = v
}
```

for RangeClause пример - Целое число (integer value)

```
n := 5
for i := range n {
    // тип i это int (так как n имеет тип int)
    // i принимает значения 0, 1, 2, 3, 4
}

n := 0
for range n {
    // тело не выполнится
    // если n <= 0 — цикл не выполнится ни разу
}

var i int8
n := 5
for i = range n {
    // тип i остаётся int8 (тип переменной)
    // если переменная итерации уже существует, тип значений итерации берётся от неё
    // значения итерации приводятся к типу переменной i
}

for i := range 3 {
    // тип i это int (default type для нетипизированной константы)
}
```

for RangeClause пример - Функция, 0 значений (Go 1.23+)

```
f := func(yield func() bool) {  
    for i := 0; i < 3; i++ {  
        // yield вызывается без аргументов, возвращает bool. Если тело цикла прервано (break), yield вернёт  
        false, и итератор должен прекратить вызовы.  
        if !yield() {  
            // Если f завершается, не вызвав yield, цикл завершается без итераций  
            return  
        }  
    }  
}  
  
for range f {  
    // тело цикла выполнится 3 раза  
    // переменных итерации нет  
  
    // Тело этого цикла не выполняется само по себе,  
    // оно становится телом функции yield, которую компилятор создаёт и передаёт в f.  
    // Если здесь написать break, то yield() вернёт false  
}
```

for RangeClause пример - Функция, 1 значений (Go 1.23+)

```
f := func(yield func(int) bool) {  
    for i := 0; i < 3; i++ {  
        // сигнатура yield-функции – func(V) bool. Значение, переданное в yield, становится значением  
        // итерации v.  
        if !yield(i) {  
            return  
        }  
    }  
}  
  
for v := range f {  
    // тип v это int  
    // v принимает значения 0, 1, 2  
}
```

for RangeClause пример - Функция, 2 значений (Go 1.23+)

```
f := func(yield func(int, string) bool) {  
    pairs := []struct {  
        k int  
        v string  
    }{{1, "a"}, {2, "b"}}  
  
    for _, p := range pairs {  
        // сигнатура yield-функции – func(K, V) bool. Первый аргумент становится первой переменной итерации,  
        // второй – второй.  
        if !yield(p.k, p.v) {  
            return  
        }  
    }  
}  
  
for k, v := range f {  
    // тип k это int  
    // тип v это string  
}
```

Потоки и стандартные устройства

- `os.Stdin` — стандартный ввод
- `os.Stdout` — стандартный вывод
- `os.Stderr` — стандартный поток ошибок
- `fmt` — пакет форматированного ввода-вывода
- `bufio` — буферизованный ввод-вывод
- `io` — общие интерфейсы `Reader` / `Writer`

Буферизация

- `os.Stdout` по умолчанию буферизуется ОС.
- `os.Stderr` обычно не буферизуется.
- В Go нет `std::endl` — вместо него `fmt.Println` (добавляет `\n`) и `fmt.Print`.
- Для сброса буфера — `Flush` у `bufio.Writer`.
- `fmt.Println` и `fmt.Printf` пишут напрямую в `os.Stdout` (при необходимости — через `syscall`).

fmt.Println — ВЫВОД

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")

    age := 20
    fmt.Println("Age:", age)

    fmt.Printf("Age: %d\n", age)
    fmt.Printf("Name: %s, pi: %.2f\n", "Bob", 3.14159)
}
```

Глаголы форматирования fmt

Глагол	Назначение
%v	значение по умолчанию
%+v	значение с именами полей (для структур)
%#v	Go-синтаксис значения
%T	тип значения
%d	целое число
%f	число с плавающей точкой
%s	строка
%q	строка в кавычках
%t	bool
%p	указатель
%x / %X	шестнадцатеричное представление
%b	двоичное представление

\n vs Println

```
fmt.Println("Hello")           // добавляет \n автоматически
fmt.Print("Hello\n")           // \n нужно указать вручную
fmt.Printf("Hello %s\n", "Bob") // форматированный вывод

fmt.Fprintln(os.Stdout, "Hello") // в конкретный io.Writer
```

В Go нет std::endl и std::flush в привычном смысле.

Используйте `fmt.Println`, `fmt.Print`, `fmt.Printf`.

Ввод — fmt.Scan и fmt.Scanln

```
var x int
var y float64
var s string

fmt.Scan(&x, &y, &s)    // через пробелы
fmt.Scanln(&x)          // до конца строки

fmt.Sprintf("%d %f %s", &x, &y, &s)
```

Scan требует указателей. Пропускает пробельные символы.

Часто удобнее bufio.Scanner.

bufio.Scanner — построчный ввод

```
scanner := bufio.NewScanner(os.Stdin)
for scanner.Scan() {
    line := scanner.Text()
    fmt.Println("line:", line)
}
if err := scanner.Err(); err != nil {
    fmt.Fprintln(os.Stderr, "error:", err)
}
```

Scanner читает по строкам (по умолчанию — до \n).

Можно настроить разделитель: `scanner.Split(bufio.ScanWords)`.

strconv – преобразование чисел и строк

```
n, err := strconv.Atoi("42")           // string -> int
s := strconv.Itoa(42)                  // int -> string

f, err := strconv.ParseFloat("3.14", 64)
b, err := strconv.ParseBool("true")

fmt.Println(n, s, f, b, err)
```

Строки

String literals (escape characters)

Expression	Result	Note
<code>" "</code>		Default zero value for type <code>string</code>
<code>"Japan 日本"</code>	Japan 日本	Go code is Unicode text encoded in UTF-8
<code>"\xe6\x97\xa5"</code>	日	<code>\xNN</code> specifies a byte
<code>"\u65E5"</code>	日	<code>\uNNNN</code> specifies a Unicode value
<code>"\""</code>	<code>\</code>	Backslash
<code>"\""</code>	<code>"</code>	Double quote
<code>"\n"</code>		Newline
<code>"\t"</code>		Tab
<code>`\xe6`</code>	<code>\xe6</code>	Raw string literal*
<code>html.EscapeString("<>")</code>	<code>&lt;&gt;</code>	HTML escape for <code><</code> , <code>></code> , <code>&</code> , <code>'</code> and <code>"</code>
<code>url.PathEscape("A B")</code>	<code>A%20B</code>	URL percent-encoding net/url

* In `` ` `` string literals, text is interpreted literally and backslashes have no special meaning. See [Escapes and multiline strings](#) for more on raw strings, escape characters and string encodings.

Строки

Concatenate

Expression	Result	Note
"Ja" + "pan"	Japan	Concatenation

Performance tips

See [3 tips for efficient string concatenation](#) for how to best use a string builder to concatenate strings without redundant copying.

Equal and compare (ignore case)

Expression	Result	Note
"Japan" == "Japan"	true	Equality
strings.EqualFold("Japan", "JAPAN")	true	Unicode case folding
"Japan" < "japan"	true	Lexicographic order

Строки

Length in bytes or runes

Expression	Result	Note
<code>len("日")</code>	3	Length in bytes
<code>utf8.RuneCountInString("日")</code>	1	in runes unicode/utf8
<code>utf8.ValidString("日")</code>	true	UTF-8? unicode/utf8

Строки

Length in bytes or runes

Expression	Result	Note
<code>len("日")</code>	3	Length in bytes
<code>utf8.RuneCountInString("日")</code>	1	in runes unicode/utf8
<code>utf8.ValidString("日")</code>	true	UTF-8? unicode/utf8

Строки

Index, substring, iterate

Expression	Result	Note
"Japan"[2]	'p'	Byte at position 2
"Japan"[1:3]	ap	Byte indexing
"Japan"[:2]	Ja	
"Japan"[2:]	pan	

A Go [range loop](#) iterates over UTF-8 encoded characters ([runes](#)):

```
for i, ch := range "Japan 日本" {  
    fmt.Printf("%d:%q ", i, ch)  
}  
// Output: 0:'J' 1:'a' 2:'p' 3:'a' 4:'n' 5:' ' 6:'日' 9:'本'
```

Iterating over bytes produces nonsense characters for non-ASCII text:

```
s := "Japan 日本"  
for i := 0; i < len(s); i++ {  
    fmt.Printf("%q ", s[i])  
}  
// Output: 'J' 'a' 'p' 'a' 'n' ' ' 'æ' '\u0097' '¥' 'æ' '\u009c' '↪'
```

Строки

Search (contains, prefix/suffix, index)

Expression	Result	Note
<code>strings.Contains("Japan", "abc")</code>	false	Is abc in Japan?
<code>strings.ContainsAny("Japan", "abc")</code>	true	Is a, b or c in Japan?
<code>strings.Count("Banana", "ana")</code>	1	Non-overlapping instances of ana
<code>strings.HasPrefix("Japan", "Ja")</code>	true	Does Japan start with Ja?
<code>strings.HasSuffix("Japan", "pan")</code>	true	Does Japan end with pan?
<code>strings.Index("Japan", "abc")</code>	-1	Index of first abc
<code>strings.IndexAny("Japan", "abc")</code>	1	a, b or c
<code>strings.LastIndex("Japan", "abc")</code>	-1	Index of last abc
<code>strings.LastIndexAny("Japan", "abc")</code>	3	a, b or c

Строки

Replace (uppercase/lowercase, trim)

Expression	Result	Note
<code>strings.Replace("foo", "o", ".", 2)</code>	<code>f..</code>	Replace first two "o" with "." Use -1 to replace all
<code>f := func(r rune) rune { return r + 1 } strings.Map(f, "ab")</code>	<code>bc</code>	Apply function to each character
<code>strings.ToUpper("Japan")</code>	<code>JAPAN</code>	Uppercase
<code>strings.ToLower("Japan")</code>	<code>japan</code>	Lowercase
<code>strings.Title("ja pan")</code>	<code>Ja Pan</code>	Initial letters to uppercase
<code>strings.TrimSpace(" foo\n")</code>	<code>foo</code>	Strip leading and trailing white space
<code>strings.Trim("foo", "fo")</code>		Strip <i>leading and trailing</i> f:s and o:s
<code>strings.TrimLeft("foo", "f")</code>	<code>oo</code>	<i>only leading</i>
<code>strings.TrimRight("foo", "o")</code>	<code>f</code>	<i>only trailing</i>
<code>strings.TrimPrefix("foo", "fo")</code>	<code>o</code>	
<code>strings.TrimSuffix("foo", "o")</code>	<code>fo</code>	

Строки

Split by space or comma

Expression	Result	Note
<code>strings.Fields(" a\t b\n")</code>	<code>["a" "b"]</code>	Remove white space
<code>strings.Split("a,b", ",")</code>	<code>["a" "b"]</code>	Remove separator
<code>strings.SplitAfter("a,b", ",")</code>	<code>["a," "b"]</code>	Keep separator

Join strings with separator

Expression	Result	Note
<code>strings.Join([]string{"a", "b"}, ":")</code>	<code>a:b</code>	Add separator
<code>strings.Repeat("da", 2)</code>	<code>dada</code>	2 copies of "da"

Строки

Format and convert

Expression	Result	Note
<code>strconv.Itoa(-42)</code>	<code>"-42"</code>	Int to string
<code>strconv.FormatInt(255, 16)</code>	<code>"ff"</code>	Base 16

Sprintf

The `fmt.Sprintf` function is often your best friend when formatting data:

```
s := fmt.Sprintf("%.4f", math.Pi) // s == "3.1416"
```

Массивы

[0]	[1]	[2]	[3]	[4]
10	20	30	40	50

Values of the array before changing

[0]	[1]	[2]	[3]	[4]
10	20	30	40	55

Values of the array after changing

Массивы

```
// Массив чисел
// Длина фиксирована и = 8
// Все элементы типа int
// Все элементы инициализированы нулевым значением
var planets [8]int
fmt.Print(planets)
```

Массивы

```
// Вначале пустой
var planets [8]string
planets[0] = "Меркурий" // Присваивает планете индекс 0
planets[1] = "Венера"
planets[2] = "Земля"

// Ещё один массив
planets := [5]string{"Церера", "Плутон", "Хаумеа", "Макемаке", "Эрида"}

// Длина определяется по количеству элементов
planets := [...]string{ // Компилятор Go подсчитывает элементы
    "Меркурий",
    "Венера",
    "Земля",
    "Марс",
    "Юпитер",
    "Сатурн",
    "Уран",
    "Нептун", // Запятая в самом конце является обязательной
}
```

Массивы

```
planets := [...]string{
    "Меркурий",
    "Венера",
    "Земля",
    "Марс",
    "Юпитер",
    "Сатурн",
    "Уран",
    "Нептун",
}
```

```
planetsMarkII := planets // Копирует массив planets
```

```
planets[2] = "упс" // Прокладывает путь для межзвездного шунтирования
```

```
fmt.Println(planets)          // Выводит: [Меркурий Венера упс Марс Юпитер Сатурн Уран Нептун]
fmt.Println(planetsMarkII)    // Выводит: [Меркурий Венера Земля Марс Юпитер Сатурн Уран Нептун]
```

Массивы

```
planets := [...]string{
    "Меркурий",
    "Венера",
    "Земля",
    "Марс",
    "Юпитер",
    "Сатурн",
    "Уран",
    "Нептун",
}

planetsMarkII := planets // Копирует массив planets

planets[2] = "упс" // Прокладывает путь для межзвездного шунтирования

fmt.Println(planets)          // Выводит: [Меркурий Венера упс Марс Юпитер Сатурн Уран Нептун]
fmt.Println(planetsMarkII)    // Выводит: [Меркурий Венера Земля Марс Юпитер Сатурн Уран Нептун]

var planetsMarkIII [8]string = planets // Тоже работает
var planetsMarkIV [10]string = planets  // Ошибка
```

Массивы многомерные

```
var board [3][3]string // Массив из 3 массивов с 3 строками
```

```
    // Координаты [ряд][столбец]
```

```
board[0][0] = "1"
```

```
board[0][1] = "2"
```

```
board[0][2] = "3"
```

```
board[1][0] = "4"
```

```
board[1][1] = "5"
```

```
board[1][2] = "6"
```

```
board[2][0] = "7"
```

```
board[2][1] = "8"
```

```
board[2][2] = "9"
```

```
for i:=0; i<3; i++ {
```

```
    fmt.Println(board[i][0], board[i][1], board[i][2])
```

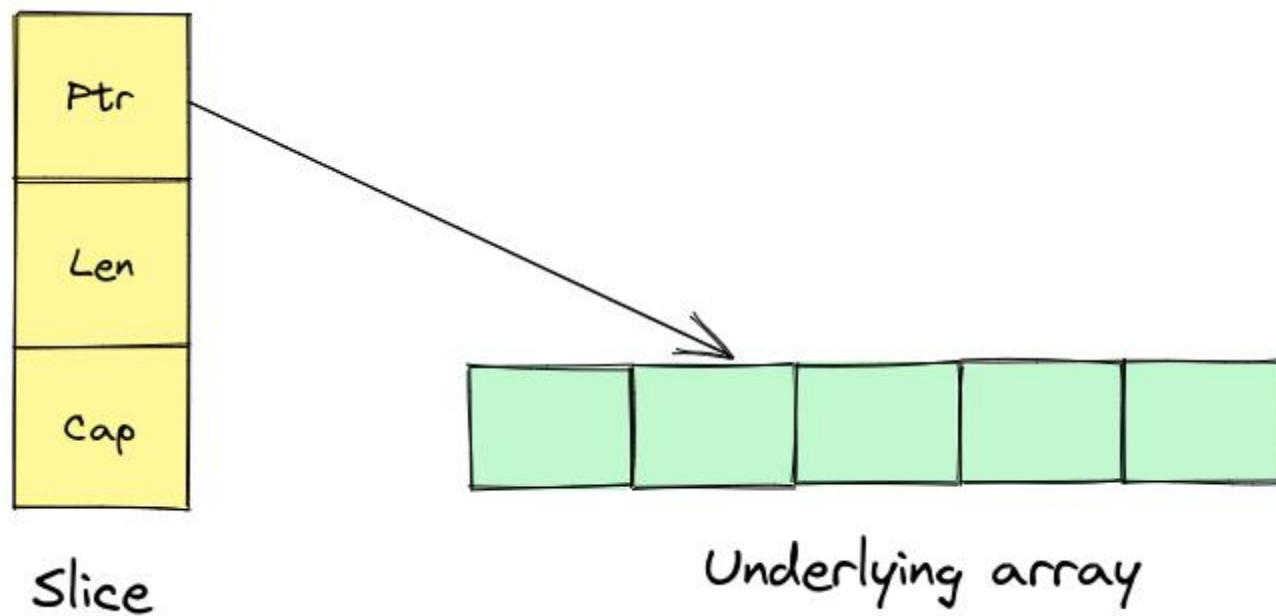
```
}
```

Массивы многомерные

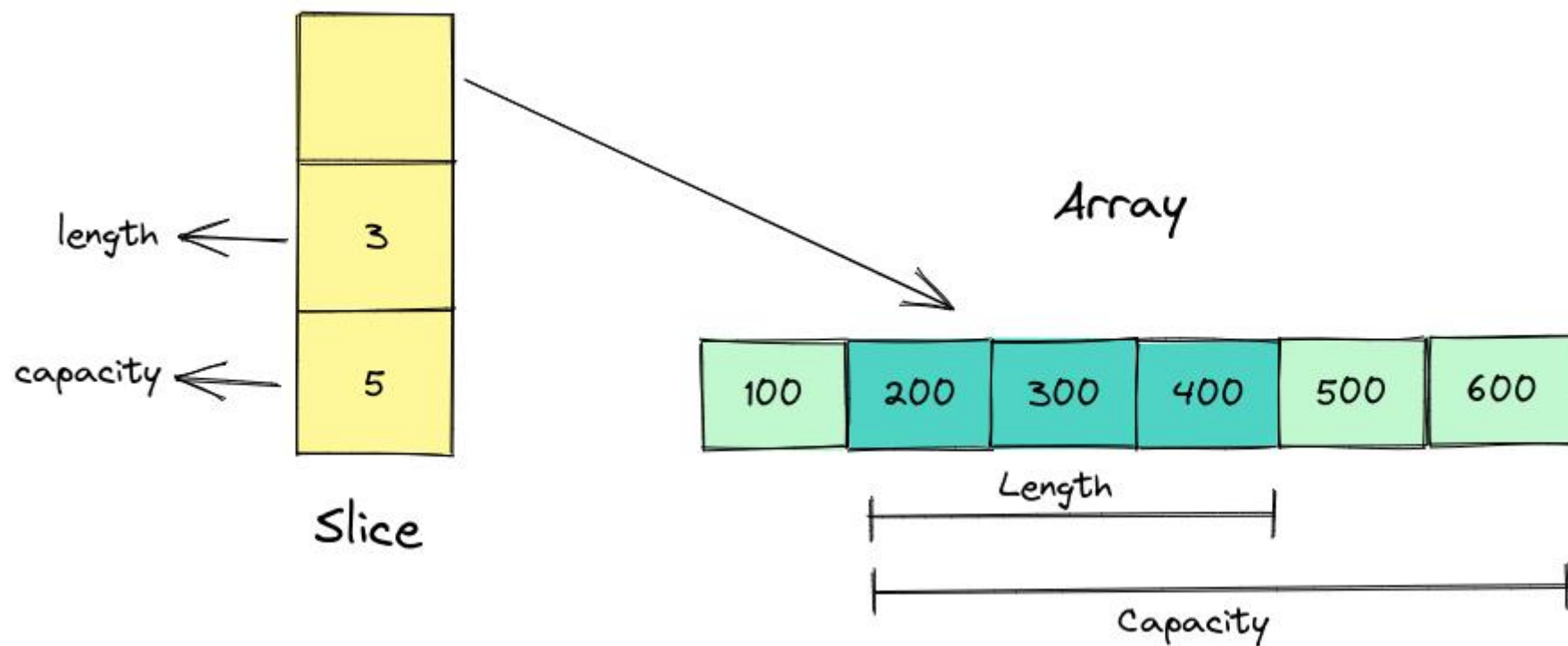
```
var board =  [3][3]int{{1,2,3},  
                      {4,5,6},  
                      {7,8,9}}
```

```
for i:=0; i<3; i++ {  
    fmt.Println(board[i][0], board[i][1], board[i][2])  
}
```

Срез (слайс)



Срез (слайс)



Слайс

```
// Срез чисел  
// Длина может изменяться  
// Все элементы типа int  
// Сейчас никуда не ссылается  
var planets []int  
fmt.Print(planets == nil)
```

Слайс

```
// make создаст новый слайс для 10 элементов и 3 из них занулит  
// Сейчас ссылается на массив  
var planets []int = make([]int, 3, 10)  
fmt.Print(planets, planets == nil)
```

Слайс

```
var planets []int = []int{1, 2, 3, 4, 5}
```

```
planetsII := []int{1, 2, 3, 4, 5}
```

```
planetsIII := planetsII
```

```
// planetsII и planetsIII ссылаются на один и тот же массив
```

```
fmt.Print(planets, planetsII, planetsIII)
```

Слайс

```
planets := [5]int{1, 2, 3, 4, 5} // Это НЕ слайс, это массив  
other := planets[0 : 2]  
fmt.Println(planets, other)
```

```
planets[0] = 10  
fmt.Println(planets, other)
```

```
other[0] = -10  
fmt.Println(planets, other)
```

Слайс

```
planets := [5]int{1, 2, 3, 4, 5} // Это НЕ слайс, это массив
other := planets[0:2]           // [1, 2]
otherI := planets[:2]           // [1, 2]
otherII := planets[2:]          // [3, 4, 5]
otherIII := planets[:]          // [1, 2, 3, 4, 5]
```

Многомерный слайс

```
seaNames := [][]string{{"shark", "octopus", "squid", "mantis shrimp"},  
                        {"Sammy", "Jesse", "Drew", "Jamie"}}
```

```
fmt.Print(seaNames)
```

Слайс – основные функции

```
planets := []int{1, 2, 3, 4, 5}

// len - текущая длина слайса
// cap - ёмкость
fmt.Println(planets, len(planets), cap(planets)) // [1, 2, 3, 4, 5] 5 5

planets = append(planets, 9)
fmt.Println(planets, len(planets), cap(planets)) // [1, 2, 3, 4, 5, 9] 6 10

newPlanets := make([]int, 3, 20)
copy(newPlanets, planets)
fmt.Println(newPlanets, len(newPlanets), cap(newPlanets)) // [1, 2, 3] 3 20

newPlanets = append(newPlanets, planets...)
fmt.Println(newPlanets, len(newPlanets), cap(newPlanets)) // [1 2 3 1 2 3 4 5 9] 9 20

// Удалим 2 элемент из planets
planets = append(planets[:2], planets[3:]...)
fmt.Println(planets) // [1, 2, 4, 5, 9]
```

<https://pkg.go.dev/slices>

Слайс – append особенности

```
array := [5]int{5, 5, 5, 5, 5}
```

```
s1 := array[:3]
```

```
fmt.Println(s1) // [5, 5, 5]
```

```
s1 = append(s1, 10)
```

```
fmt.Println(array, s1) // [5, 5, 5, 10, 5] [5, 5, 5, 10]
```

```
s1 = append(s1, 11, 12) // Слайс переполнится и создаст другой массив для себя
```

```
fmt.Println(array, s1) // [5, 5, 5, 10, 5] [5, 5, 5, 10, 11, 12]
```